实验 2 异步事件与 fork/exec 的探索与实现

截止时间：

本次实验分为两个阶段提交，截止时间如下

- 第一阶段（异步事件）：2026 年 4 月 12 日 23:59
- 第二阶段（fork/exec）：2026 年 5 月 27 日 23:59

提交要求：

- 正常提交→OBE 上“实验 2-第一阶段”、“实验 2-第二阶段”；
- 截止后覆盖提交→OBE 上新开条目（待 OBE 上创建）。

实验报告编制要求：

- 文件名（**强制要求**）：学号姓名.pdf ←注意
  - **必须使用 PDF 格式，不能是 doc、docx、odt、md 等格式**
- 报告标题：体现出“实验 2-第 x 阶段”等信息（两个阶段同时提交，可以用一个 PDF 文件，但在报告开篇阐明本次提交包含两个阶段的内容）
- 报告作者：学号 姓名 ←注意
- 不同题目之间有明显的间隔，要能够容易地区分不同题目对应的实验内容
- 实验报告不做任何其他版式上的要求

提交要求：

- **提交完整可编译运行的源代码**（提交前在 compile 目录执行 make clean 清楚二进制文件以减少体积）
  - **所有代码在提交前**必须保证能在以下提供的服务器上编译通过，在非 x86 平台做实验的同学，将 Makefile 中被修改的涉及 x86、gcc 等相关命令的行（记为 O）注释掉（增加自己平台所用的特殊的命令相关的行，记为 N），在服务器上将 N 全部注释掉（不要将新增的与平台、工具无关、仅与你实现的功能相关的行注释掉——如果同时也涉及了特定的平台、工具，应提供 x86、gcc 版本），恢复 O，然后能够正常编译通过，就可以打包提交了。最后测评打分的时候用的是与服务器一致的环境（只有 x86、gcc 环境）。**如果测评打分的**

时候代码无法在该环境下编译通过，直接判定为无效提交。

- 服务器信息：ip 地址 10.10.21.42，端口号：2002，用户名：u 学号，初始密码：学号后 6 位。首次登录后尽快修改密码。
- 服务器只用于确保代码可正常编译，性能有限，不建议作为开发平台，且建议错峰使用。
- 服务器只能通过校内访问或者 VPN 接入校园网之后访问；如果无法访问，在微信群反馈。

➢ 实验报告与源码目录一起压缩，压缩包必须使用 Linux 常用的压缩包格式.tar.gz，源码不用额外打包，源码目录与实验报告一起打包。

- 假设源码目录包 Xinu-qemu-2026 和实验报告 123456 学生.pdf 在同一个目录
- 在该目录中执行：`tar czvf 123456.tar.gz .`
- 压缩包文件名为学号，上述命令最后有一个“.”

代码编写要求：

➢ 函数命名规则

- 所有新增的内核态函数均以“k 学号_”作为前缀，例：学号为 123456 的同学实现了一个内核态 printf 函数，则该函数的函数名应为“k123456_printf”
- 所有新增的用户态函数均以“u 学号_”作为前缀，例：学号为 123456 的同学实现了一个用户态 printf 函数，则该函数的函数名应为“u123456_printf”
- 在本实验手册内提及的新函数均应按照上述要求命名
- 此处所指的函数包括 C 语言函数和汇编代码函数

➢ 全局变量、结构体命名规则

- 所有新增的全局变量和结构体的命名也遵循函数命名规则
- 局部变量不用遵循上述规则

- 在 shell 目录下新建一个文件 xsh_lab2.c，其中包含一个 shell 命令函数“xsh_lab2”，参数类型请参考其他 shell 命令，本实验中所有代码均实现在该源码文件中（其他文件仅涉及非常少的修改），xsh_lab2 函数在返回前输出学号和姓名（若中文无法正常显示，可使用拼音）
- 在 include/shprototypes.h 中添加上述新增 shell 命令函数的声明
- 在 shell/shell.c 的 cmdtab 数组中，添加{“lab2”, FALSE, xsh_lab2}，注意保证语法正确(后续可在 Xinu 的 xsh 中通过 lab2 命令来测试所有题目）
- 第 2 阶段的 xsh_lab2.c 可以不包含第 1 阶段的内容，也可以包含

本次实验要求提交代码，也需提交实验报告。实验报告应说明对代码的修改（新增、修改等），以及丰富的截图和配套的文字说明（不能只有截图而无文字说明，或只有文字而无截图）。

1. 实验目的

- 希望学生探索异步事件的实现方式，了解其机理。
- 希望学生能掌握 fork/exec 的实现和调用，将 Xinu 改造成创建新进程方面更贴近现代 Linux 系统的形式。

2. 实验要求

以下 2.1 对应第 1 阶段，2.2 对应第 2 阶段。

2.1 实现一个异步的延时函数 `delay_run(int seconds, void* func, …)`，要求

a 该函数将目标函数 func 延时多少秒后执行，但该函数（`delay_run`）本身不会像 `sleep()`那样阻塞当前进程

b 形参中的“…”为 func 执行所需的参数（假设都是 int 类型，且最多 5 个）

c 在 `xsh_lab2()`中，执行对 `delay_run` 的测试，如下（请自行保证代码的正确性）

```
printf("xsh_lab2(1): %d.%d", clktime, count1000);
//注意:  此处 printf 是系统自有的
delay_run(X, delay_test, a, b, c);
// X 为延迟的秒数(以获得可观测的延迟),请自设;delay_test 对应参数 func;
后边的 a,b,c 为传递给它的参数, 请自设
// 再重复调用 2 次 delay_run, 每次使用不同的延时和参数
printf("xsh_lab2(2): %d.%d", clktime, count1000);
```

d 实现一个 delay_test 函数，它将打印传入的参数（与 2.1 中的 printf 相似，可以调用 2.1 中实现的函数来完成这里的打印参数的操作，也可直接在 delay_test 中实现），并在所有这些打印输出之前，执行如下代码（即，以下代码位于 delay_test 函数的头部）

```
printf("delay_test: %d.%d", clktime, count1000);
```

e 观察记录运行结果，并分析是否符合期望结果

f 注意：需要考虑，对所提交的代码进行测试时，可能提供其他的代码实现所谓你实现的 delay_test 的替代，应保证在这种情况下仍然能够正常实现延迟制定时间后执行 delay_test 的功能。如实验 1 中的 sndA、sndB 之类的函数应该不做修改也能通过 delay_run 延迟执行。比如，所提供的 delay_test 只包含一条代码：

```
void delay_test(int a){
    printf("delay_test: %d.%d", clktime, count1000);
}
```

2.2 将创建进程的过程修改为 fork/exec 的模式：

a 实现两个 Xinu 系统调用 int fork(void)和 void exec(void*, pri16, char*, uint32, …)

b 当进程 A 在函数 foo()中调用 fork 后，创建出进程 B，在 fork 返回时，B 是就绪或正在运行状态（不是 create 创建的新进程那种挂起状态）；

c fork 不接受参数，B 进程的栈大小可以设置默认值（如 64KB、8KB 等），也可以与父进程 A 保持一致；

d fork 返回 0 时，表示在 B 进程的上下文中，此时如果执行 exec，则切换到指定的程序入口（如 sndA、sndB、delay_test 等）去执行；如果不执行 exec，则 B 进程直接沿着 foo()函数中调用 fork 之后的代码继续执行（相当于复制了一个 A 进程）；

e fork 返回正数（>0）时，表明在 A 进程的上下文中，返回值为 B 的进程 ID；

f fork 返回负数时，表明创建子进程失败；

g exec 函数接受至少 4 个参数：新进程入口函数地址（与 create 函数参数 funcaddr 一致）、新进程的优先级（create 函数的 priority 参数）、新进程的名称（create 函数的 name 参数）、传递给入口函数的参数的数目（create 函数的 nargs 参数），即：除了栈大小 ssize 参数外，create 函数的其他参数都出现在 exec 中；

h exec 不会创建全新进程，而是在当前的 B 进程的上下文中设置栈并跳转到 funcaddr 执行；exec 正常执行的话不会返回到调用点；

i fork 和 exec 的使用方式可参考课件第 4 章第 48 页，但父进程无需使用 waitpid 之类的方式等待子进程结束；

j 在 xsh_lab2.c 中实现个测试函数；

k 测试函数 1 的架构如下（只使用 fork，不使用 exec），父子进程从 fork 返回后都输出当前正在执行的进程的 PID（返回值和 currpid 相对对照）和进程名称。（自行调整代码，以确保结果正确并排除创建子进程失败的影响）；

```
int pid = fork();
printf("curr proc: %d(%d), %s\n", pid, currpid, 进程名称);
```

l 测试函数 2 的结构如下（使用 fork 和 exec），其中，exec 的目标程序中至少应输出当前进程的 pid 和进程名，最后一个输出 currpid 的行用于检查子进程是否从 exec 返回了（正常来说，子进程是不会返回然后执行该 printf 的）。

```
int pid = fork();
if (子进程) {
    exec(目标程序, 其他参数) // 目标程序需自行提供
} else if (父进程) {
    printf("parent proc: %d(%d), %s\n", pid, currpid, 进程名称);
}
printf("currpid = %d\n", currpid);
```

3. 思维扩展

（不要求在实验报告中作答）

3.1 探索尝试实现操作系统课程中介绍的多级反馈队列轮转调度算法。

3.2 如果一个进程主动让出 CPU（如通过 sleep），下一个新进程（假设不包含任何主动让出 CPU 的操作）是否真的能运行满一个时间片？

3.3 深入分析 HarmonyOS（LiteOS-m/LiteOS-a）和 xv6 的代码，查看它们怎么处理进程延迟和进程创建，与 Xinu 有何异同。